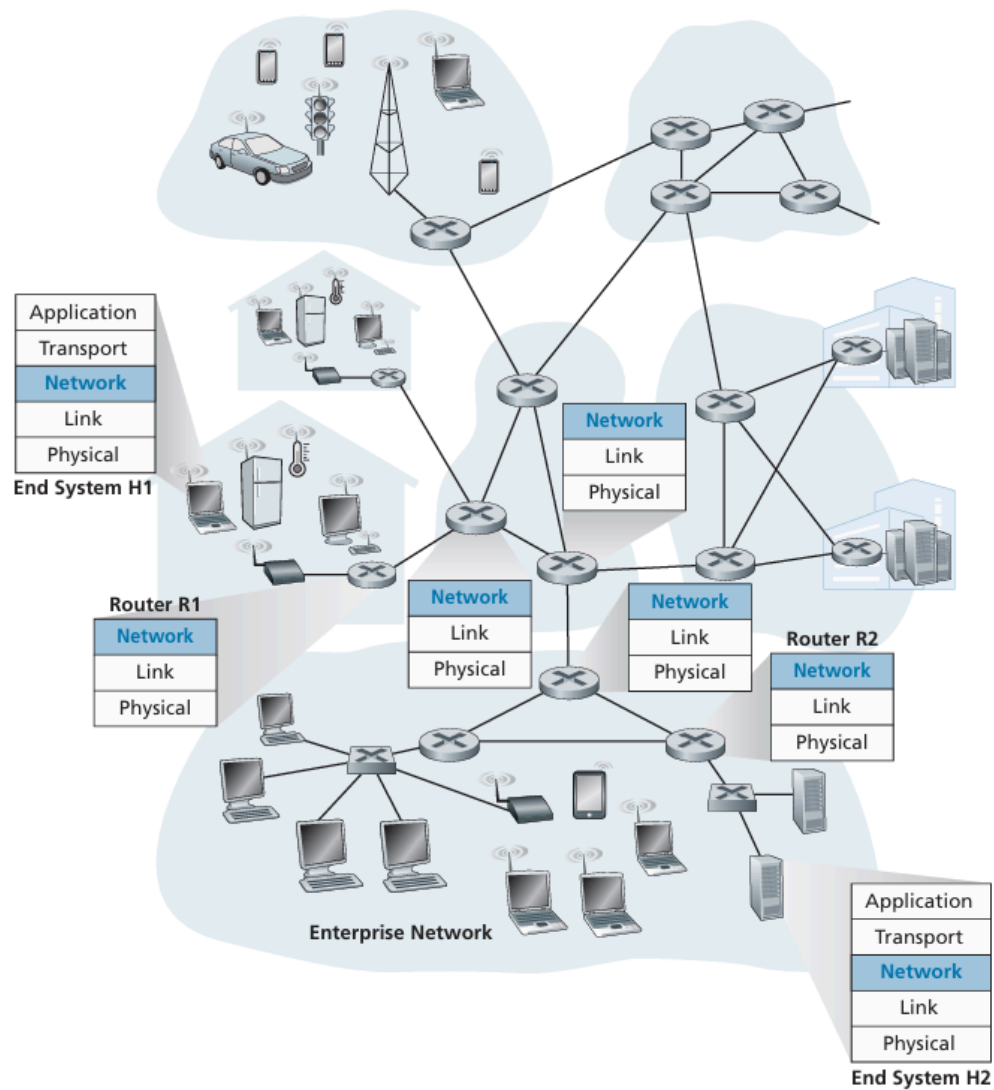


Network layer



4.1

The Network Layer

Primary Role

- The **network layer** is responsible for delivering packets from a **source host** to a **destination host** across the network.

- Though the task sounds simple, it involves critical processes to ensure that data reaches its correct destination efficiently and securely.
-

Core Functions of the Network Layer

1. Forwarding (Data Plane Function)

- Definition: The process of transferring an incoming packet from a router's **input link** to the correct **output link**.
- Example: In Figure 4.1, a packet from **Host H1** enters **Router R1**. The router must forward it to the next router on the path toward **Host H2**.
- Other possible actions:
 - Block packets (e.g., from a malicious sender or directed to a forbidden host).
 - Duplicate packets to multiple output links (multicasting).
- Characteristics:
 - Happens very quickly, within **nanoseconds**.
 - Implemented in **hardware** for speed and efficiency.
- Analogy: Like navigating a **traffic interchange**. A car enters from one road and must choose the correct exit road.

2. Routing (Control Plane Function)

- Definition: The process of determining the **path or route** that packets take across the network from the sender to the receiver.
- Uses **routing algorithms** to compute efficient and reliable paths.
- Example: The route chosen for packets from **H1 to H2** in Figure 4.1 is determined by a routing algorithm.
- Characteristics:

- Operates on a **much longer timescale** (seconds).
 - Often implemented in **software** since route calculation is more complex.
 - Analogy: Like **planning a road trip**. Before traveling from Pennsylvania to Florida, the driver consults a map and chooses a suitable route among many possible paths.
-

Forwarding vs Routing

- **Forwarding:**
 - A **local action** performed at each router.
 - Focused on the immediate step: moving a packet from input → output link.
 - Very fast and hardware-based.
 - **Routing:**
 - A **network-wide process**.
 - Decides the entire end-to-end path for packet delivery.
 - Slower, more complex, and typically software-based.
-

Forwarding Table in Routers

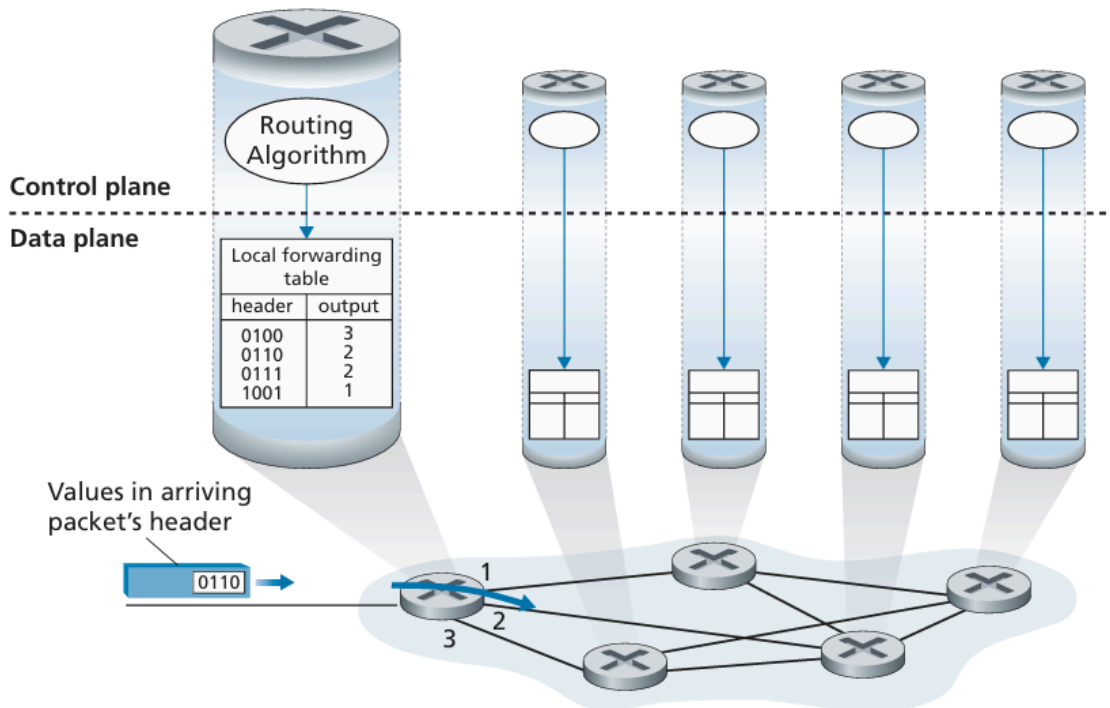
- Each router maintains a **forwarding table**, which guides the forwarding process.
- Steps:
 1. Router inspects the header of the incoming packet.
 2. The header values are used to **index into the forwarding table**.
 3. The table entry specifies the **output link/interface** for the packet.

- Example: In Figure 4.2, if a packet arrives with header value **0110**, the router consults the forwarding table and finds that it should be sent out through **interface 2**.

Summary

- The **network layer** provides two tightly linked functions:
 - **Forwarding** → fast, router-level action, ensures the packet moves hop-by-hop.
 - **Routing** → slower, global decision-making, ensures packets find a complete path.
 - Together, they make sure packets travel efficiently and reliably from **source to destination**.
-

Do you want me to keep future ones in this **sectioned format with extra explanations + examples + analogies** (like this), or should I make them even **longer, like near-notes style**?



Configuration of Forwarding Tables

Key Question

- How are **forwarding tables** in routers configured?
 - This question highlights the interplay between **forwarding (data plane)** and **routing (control plane)**.
-

Role of Routing in Forwarding Table Creation

- **Forwarding**: Moves packets from input link → output link (fast, local action).
 - **Routing**: Decides the correct paths and fills in the forwarding tables.
 - In **Figure 4.2**:
 - Each router runs a **routing algorithm**.
 - The routing algorithm determines the content of its **local forwarding table**.
 - Example: An incoming packet with header value **0110** is mapped by the table to **output 2**.
-

Communication Between Routers

- Routing algorithms in different routers communicate with each other.
 - They exchange **routing messages** that contain network topology and path information.
 - These exchanges follow **routing protocols** (covered in detail in Sections 5.2–5.4).
 - Through this process, routers cooperatively compute and update their forwarding tables.
-

Hypothetical Scenario: Manual Configuration

- Imagine if **human network operators** directly configured forwarding tables at each router:
 - No routing protocols would be needed.
 - Operators would have to coordinate to ensure correct end-to-end delivery.
 - Problems:
 - **Error-prone**: Higher chance of mistakes.
 - **Slow adaptation**: Human configuration would be too slow to react to failures or topology changes.
 - This scenario highlights why **automated routing protocols** are essential.
-

Summary

- **Forwarding tables** are not static; they are generated and updated by the **routing function**.
 - **Routing algorithms** compute the correct paths and fill forwarding tables accordingly.
 - **Routing protocols** enable routers to exchange information and stay synchronized.
 - Manual configuration is technically possible but impractical due to slowness and errors.
 - Therefore, both **forwarding (fast packet movement)** and **routing (path decision-making)** are necessary for efficient network operation.
-

Control Plane: The SDN Approach

Traditional Approach (Figure 4.2)

- Each router contains both:
 - **Data Plane** → forwarding function.
 - **Control Plane** → routing algorithm that communicates with routing algorithms in other routers.
 - This distributed approach has been the standard for routing vendors.
 - Limitation: tightly couples routing decisions with physical routers.
-

Alternative: Centralized Control Plane (Figure 4.3)

- A **remote controller** is physically separated from routers.
 - Roles are divided:
 - Routers: perform **forwarding only** (data plane).
 - Remote Controller: computes and distributes **forwarding tables** (control plane).
 - Data plane remains identical to the traditional model—only the control plane is moved.
 - The controller may be located in a **remote data center** with redundancy and reliability.
 - Communication occurs via **messages** that carry forwarding tables and routing information between controller and routers.
-

Software-Defined Networking (SDN)

- This centralized approach is the foundation of **SDN** (Software-Defined Networking).
- Why “software-defined”?
 - The controller is implemented in **software**, not hardcoded into each router.
 - The controller dictates forwarding behavior by installing rules into routers.

- Benefits:
 - Greater **flexibility**: The control software can be modified or extended.
 - Increasingly **open-source**: Many SDN controllers are publicly available, similar to Linux OS code.
 - Enables **innovation**: ISPs, researchers, and students can experiment with and improve network-layer functionality.
 - High **reliability** and **scalability** when managed in modern data centers.
-

Summary

- **Traditional model**: routing and forwarding combined within each router (distributed control plane).
 - **SDN model**: forwarding stays in routers, but routing control shifts to a **centralized software controller**.
 - This separation makes networks more programmable, flexible, and open to innovation.
 - SDN represents a major shift in how the **control plane** is designed and deployed.
-

Input Port Processing and Forwarding Table Lookup

Role of Input Ports

- Perform **physical-layer** and **link-layer** processing for incoming links.
- Central task: **lookup function**, where the forwarding table is consulted to determine the correct output port.
- Forwarding table:

- May be computed by the **routing processor** (using routing protocols).
- Or received from a **remote SDN controller**.
- To avoid bottlenecks:
 - Forwarding tables are copied (“shadow copy”) onto each **line card**.
 - Enables local lookup at input ports instead of consulting the central routing processor for each packet.

Scaling Problem with IP Addresses

- IPv4 has **32-bit addresses** → over **4 billion possible destinations**.
- Brute-force forwarding table (one entry per address) is impossible.
- Solution: **Prefix-Based Forwarding**

Destination Address Range	Link Interface
11001000 00010111 00010000 00000000 through 11001000 00010111 00010111 11111111	0
11001000 00010111 00011000 00000000 through 11001000 00010111 00011000 11111111	1
11001000 00010111 00011001 00000000 through 11001000 00010111 00011111 11111111	2
Otherwise	3

Clearly, for this example, it is not necessary to have 4 billion entries in the router’s forwarding table. We could, for example, have the following forwarding table with just four entries:

Prefix	Link Interface
11001000 00010111 00010	0
11001000 00010111 00011000	1
11001000 00010111 00011	2
Otherwise	3

- Forwarding entries are grouped by address **prefix ranges** instead of individual addresses.
- Example (4 links, interfaces 0–3):

- Destination address ranges grouped into just 4 prefix entries.
 - Router forwards packet to the link that matches the **longest prefix**.
 - Example lookup:
 - Address 11001000 00010111 00010110 10100001 matches first prefix (21 bits). → Forward to **interface 0**.
 - If no match → forward to default interface.
-

Longest Prefix Matching Rule

- Problem: a destination may match **multiple prefixes**.
 - Solution: use the **longest prefix match** to ensure the most specific route is chosen.
 - Example:
 - Address matches both a 21-bit prefix (interface 2) and a 24-bit prefix (interface 1).
 - Router selects the 24-bit match since it's more specific.
-

High-Speed Lookup Challenges

- Lookup must be performed in **nanoseconds** at gigabit rates.
- Example: 10 Gbps link + 64-byte packet → very little time per packet.
- Techniques:
 - Implemented in **hardware**.
 - Use advanced **fast lookup algorithms** [Gupta 2001; Ruiz-Sanchez 2001].
 - Specialized memory:

- **On-chip DRAM + SRAM caches** for speed.
 - **Ternary Content Addressable Memories (TCAMs)**: allow direct lookup of 32-bit IP addresses with near-constant time.
 - Cisco routers can store **millions of TCAM entries**.
-

After Lookup

- Once output port is determined, the packet is sent to the **switching fabric**.
 - If fabric is busy, packets are queued at the input port until scheduled.
-

Additional Input Port Processing

1. **Physical & Link-layer processing**.
 2. **Header verification**: version number, checksum, and time-to-live (TTL).
 3. **Header updates**: checksum and TTL must be rewritten.
 4. **Management counters**: update packet statistics (e.g., number of datagrams received).
-

Match + Action Abstraction

- Input processing is an instance of a broader “**match + action**” **model** used in many devices:
 - **Routers**: match destination IP → forward to output port.
 - **Switches**: match link-layer destination → forward frame.
 - **Firewalls**: match specific headers → drop packet.
 - **NATs**: match transport port → rewrite header fields.

- This abstraction is **flexible, powerful, and central to generalized forwarding**.

-

IPv4 Datagram Format

Overview

- The **IPv4 datagram** is the fundamental packet structure in the Internet.
- It contains both a **header** (control information) and **data** (payload).
- Standard header size: **20 bytes** (without options).

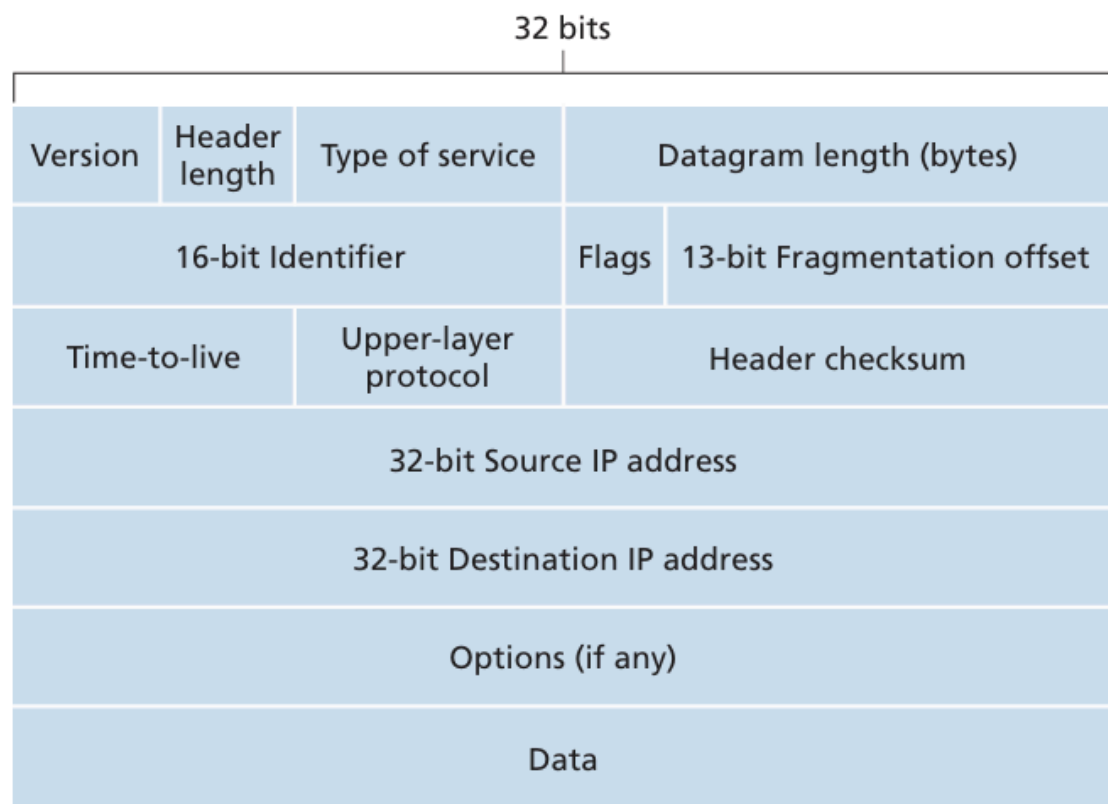


Figure 4.17 ♦ IPv4 datagram format

Key Fields in IPv4 Datagram

1. Version Number (4 bits)

- Specifies the IP version (e.g., IPv4, IPv6).
- Tells the router how to interpret the rest of the datagram.

2. Header Length (4 bits)

- Indicates where the **payload** begins.
- Needed because headers can have variable length (if options are included).
- Typical IPv4 header = **20 bytes** (no options).

3. Type of Service (TOS)

- Differentiates types of traffic (e.g., real-time vs. bulk transfer).
- Includes bits for **Explicit Congestion Notification (ECN)**.

4. Datagram Length (16 bits)

- Total size of datagram (header + data).
- Maximum: **65,535 bytes**.
- Typical: $\leq 1,500$ **bytes** (to fit inside Ethernet frame).

5. Identifier, Flags, Fragmentation Offset

- Used for **IP fragmentation and reassembly**.
- Allows large datagrams to be split into smaller ones for transmission.
- IPv6 eliminates fragmentation at routers.

6. Time-to-Live (TTL)

- Ensures datagrams do not circulate forever (e.g., in routing loops).
- Decrement by 1 at each hop; dropped if TTL = 0.

7. Protocol

- Indicates which **transport-layer protocol** the data belongs to.
- Examples:
 - 6 = TCP
 - 17 = UDP
- Serves as the "glue" between **network layer and transport layer**.

8. Header Checksum

- Used for **error detection** of the header only.
- Calculated using 1's complement arithmetic.
- Must be recomputed at each hop since TTL and possibly options change.
- If mismatch detected → datagram discarded.
- Complements transport-layer checksum (which covers full segment).

9. Source and Destination IP Addresses (32 bits each)

- **Source:** IP address of the sender.
- **Destination:** ultimate recipient's IP address (often determined via DNS).

10. Options (if any)

- Rarely used; allow header extension.
- Variable length → complicates router processing and slows performance.
- Not included in IPv6 header.

11. Data (Payload)

- The actual content, usually a **transport-layer segment** (TCP/UDP).
- Can also carry ICMP messages or other control data.

- Example: if carrying TCP → total overhead = **40 bytes** (20 for IP + 20 for TCP).
-

Important Notes

- **Overhead:** Even before application data is sent, headers consume bandwidth.
 - **Processing impact:** Options and checksum recalculations increase router workload.
 - **Layer binding:**
 - Protocol field binds **network** → **transport**.
 - Port numbers bind **transport** → **application**.
 - **Design trade-off:** IPv4 header is flexible but complex → IPv6 simplifies by removing options and fragmentation at routers.
-

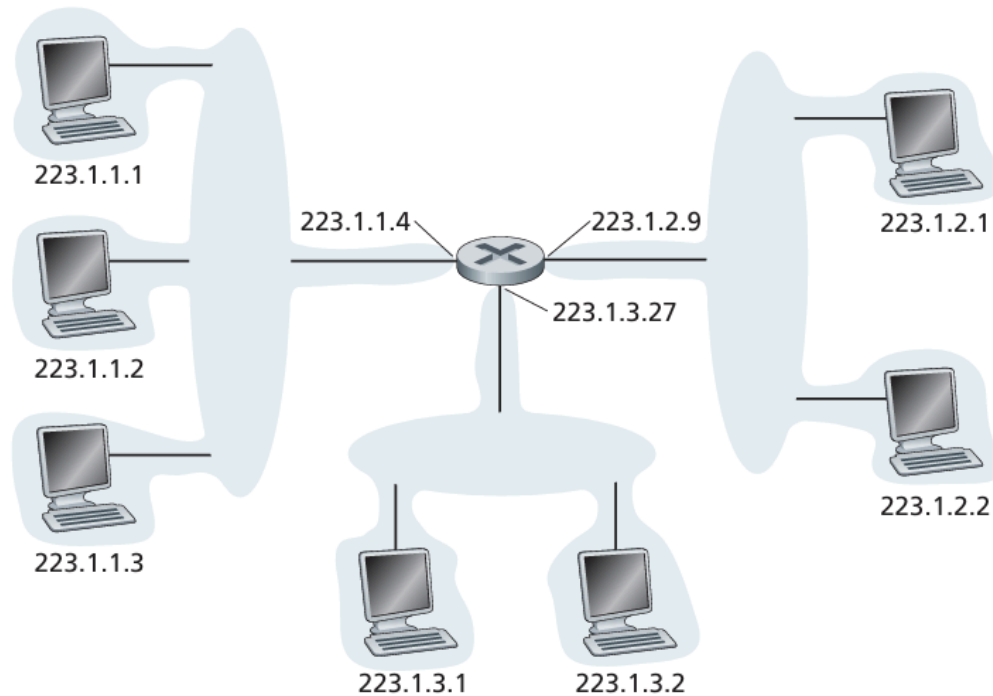
Summary

- IPv4 datagram = **header + data**.
- Header carries essential control info: addressing, delivery rules, error detection, fragmentation, and protocol type.
- Data field carries the actual message.
- Despite limitations, IPv4 remains central to Internet operations, though IPv6 is designed to simplify and extend these capabilities.

IPv4 Addressing and Subnets

Interfaces and IP Addresses

- An **interface** = the boundary between a host/router and its physical link.
-



- **Hosts** typically have 1 interface; **routers** have multiple interfaces (one per link).
- Each **interface** must have a unique **IP address**.
- IP address length = **32 bits (4 bytes)** → approx. **4 billion possible addresses**.
- Written in **dotted-decimal notation** (e.g., 193.32.216.9 = 11000001 00100000 11011000 00001001).

IP Address Association

- IP addresses are assigned to **interfaces**, not to whole hosts or routers.
- Every interface in the Internet must have a **globally unique IP address** (except private/NAT cases).

Subnets

- A **subnet** = a group of interfaces with the same high-order bits in their IP addresses and connected without intervening routers.
 - Defined using **subnet masks** in the form `/N` where **N** = number of fixed (network) bits.
 - Example: `/24` means first 24 bits define the subnet.
-

Example: Figure 4.18 Explanation

- **Subnet 1: 223.1.1.0/24**
 - Hosts: 223.1.1.1, 223.1.1.2, 223.1.1.3.
 - Router interface: 223.1.1.4.
 - All share first 24 bits (223.1.1.xxx).
 - **Subnet 2: 223.1.2.0/24**
 - Hosts: 223.1.2.1, 223.1.2.2.
 - Router interface: 223.1.2.9.
 - Address form: 223.1.2.xxx.
 - **Subnet 3: 223.1.3.0/24**
 - Hosts: 223.1.3.1, 223.1.3.2.
 - Router interface: 223.1.3.27.
 - Address form: 223.1.3.xxx.
-

Key Points

- Each subnet groups hosts + router interfaces sharing the same **prefix**.

- Router connects multiple subnets by having **different IP addresses on each interface**.
 - Notation `/24` shows how many bits belong to the **network prefix**.
 - Hosts in the same subnet communicate directly (no router needed).
 - Communication **across subnets** requires a router.
-

✓ In this figure, the single router interconnects **three subnets**: `223.1.1.0/24`, `223.1.2.0/24`, and `223.1.3.0/24`.

Do you want me to also make a **visualized subnet table** (Subnet → Members → Router Interface → Address Range) for this example so it's easier to revise quickly?

CIDR, Route Aggregation, and Specific Routes

Address Aggregation (Route Summarization)

- **CIDR (Classless Interdomain Routing)** allows ISPs to advertise a **single prefix** that represents multiple organizations.
 - Example (Figure 4.21):
 - Fly-By-Night-ISP connects **eight organizations** with address blocks inside **200.23.16.0/20**.
 - To the rest of the Internet, it advertises only **200.23.16.0/20**.
 - External routers need not know about each organization's finer subnetting.
 - This reduces the size of global forwarding tables → more efficient routing.
-

When Hierarchy Breaks

- Aggregation works best when addresses are allocated **hierarchically** (blocks from ISPs to organizations).
 - But issues arise when ownership or structure changes:
 - Example: Fly-By-Night-ISP buys **ISPs-R-Us**.
 - ISPs-R-Us owns **199.31.0.0/16**, but **Organization 1** (200.23.18.0/23) is outside that range.
 - Solution options:
 - **Renumber Org 1** into ISPs-R-Us's block → costly and disruptive.
 - **Keep Org 1's block** and advertise it separately.
-

Longest Prefix Matching in Action (Figure 4.22)

- Fly-By-Night-ISP continues to advertise **200.23.16.0/20**.
 - ISPs-R-Us advertises:
 - Its own **199.31.0.0/16**.
 - Additionally, **200.23.18.0/23** for Organization 1.
 - Routers in the Internet choose **longest prefix match**:
 - If destination is in **200.23.18.0/23**, route goes to **ISPs-R-Us** (more specific prefix).
 - If destination is in other parts of **200.23.16.0/20**, route goes to **Fly-By-Night-ISP**.
-

Broadcast Address

- **255.255.255.255** = special broadcast address.
- A datagram sent here is delivered to **all hosts in the same subnet**.

- Routers usually **do not forward broadcasts** beyond a subnet (to avoid flooding).

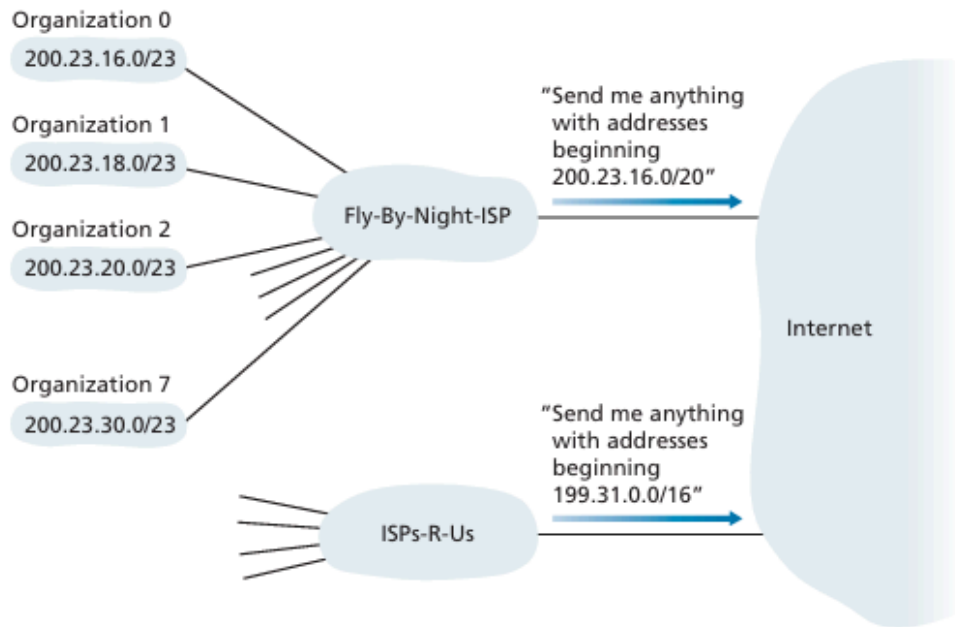


Figure 4.21 ♦ Hierarchical addressing and route aggregation

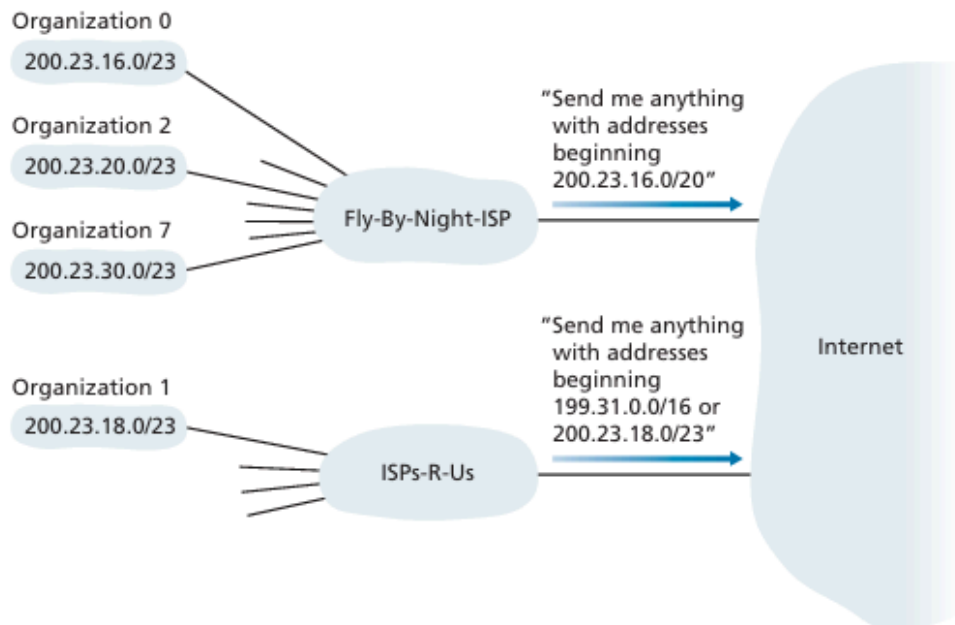


Figure 4.22 ♦ ISPs-R-Us has a more specific route to Organization 1

Summary

- **CIDR** enables efficient aggregation by allowing ISPs to advertise one block for many organizations.
 - **Route aggregation** reduces the number of global routing entries.
 - If an organization's address block doesn't align with ISP hierarchy, **specific prefixes** must still be advertised.
 - **Longest prefix matching** ensures packets follow the most precise route.
 - Special case: **broadcast address** delivers datagrams to all hosts in a subnet.
-

👉 Would you like me to also create a **comparison chart** (Aggregation vs Specific Prefix Advertisement) showing **advantages, disadvantages, and when each occurs?**

Obtaining IP Address Blocks

Step 1: From ISP to Organization

- An organization typically requests IP addresses from its **ISP**.
- ISP owns a larger allocated block and subdivides it into smaller chunks.
- Example:
 - ISP's block = **200.23.16.0/20**.
 - ISP divides into **eight /23 sub-blocks**, each given to one organization:
 - Org 0 → **200.23.16.0/23**
 - Org 1 → **200.23.18.0/23**

- Org 2 → **200.23.20.0/23**
 - ...
 - Org 7 → **200.23.30.0/23**
- Each organization then assigns addresses from its block to its own **hosts and routers**.
-

Step 2: Where ISPs Get Addresses

- ISPs themselves need to obtain larger blocks of addresses.
 - This is managed by a **global authority**:
 - **ICANN (Internet Corporation for Assigned Names and Numbers)**.
 - ICANN oversees IP address allocation and DNS root server management.
 - Also manages domain names and resolves disputes.
-

Step 3: Regional Internet Registries (RIRs)

- ICANN does not allocate directly to every ISP.
- Instead, ICANN allocates large address blocks to **Regional Internet Registries (RIRs)**.
- Examples of RIRs:
 - **ARIN** (American Registry for Internet Numbers) – North America.
 - **RIPE NCC** – Europe, Middle East, parts of Central Asia.
 - **APNIC** – Asia Pacific.
 - **LACNIC** – Latin America and Caribbean.
 - (Together these form the **Address Supporting Organization (ASO)** within ICANN.)

- RIRs then allocate smaller blocks to ISPs, who further subdivide and assign to organizations.

Summary

- **Organizations** → request address blocks from their **ISP**.
- **ISPs** → obtain larger blocks from **RIRs**.
- **RIRs** → get allocations from **ICANN**, which oversees the global IP address space.
- This hierarchical system ensures **efficient, non-overlapping, globally unique IP address allocation**.

👉 Do you want me to also make a **hierarchical diagram (ICANN → RIRs → ISPs → Organizations → Hosts)** so you can instantly visualize this flow?

Dynamic Host Configuration Protocol (DHCP) – Detailed Explanation

Why DHCP is Needed

- Every device (host) connecting to a network **needs an IP address** and configuration to communicate.
- Without DHCP:
 - Admins would have to **manually configure** each host with IP, subnet mask, gateway, DNS, etc.
 - This becomes impossible in **large or mobile environments** (e.g., laptops moving between Wi-Fi hotspots).

- With DHCP:
 - Host configuration is **automatic and fast**.
 - Known as **plug-and-play / zero-configuration (zeroconf)**.
 - Widely used in **residential Internet, enterprise LANs, and Wi-Fi networks**.
-

What DHCP Provides to a Host

When a client joins a subnet, DHCP can assign:

1. **IP Address** → either fixed (always the same) or dynamic (different each time).
 2. **Subnet Mask** → defines the host's subnet (e.g., */24*).
 3. **Default Gateway** → IP of first-hop router for external communication.
 4. **DNS Server Address** → IP address of DNS resolver for domain lookups.
 5. **Lease Time** → how long the IP is valid before renewal is required.
-

The DHCP Client-Server Model

- **DHCP Client:** Newly arriving host (e.g., laptop, phone).
- **DHCP Server:** Provides IP addresses and configuration.
- **DHCP Relay Agent:** A router that forwards DHCP messages between clients and servers if no server exists in the same subnet.
 - Example: In **Figure 4.23**, server is in subnet *223.1.2/24*.
 - Clients in subnets *223.1.1/24* or *223.1.3/24* rely on the router as relay agent.

The 4-Step DHCP Process (DORA)

Shown in **Figure 4.24**:

1. DHCP Discover (Client → Broadcast)

- Client has no IP yet.
- Constructs a DHCP Discover message:
 - **Source IP:** `0.0.0.0` (unknown).
 - **Destination IP:** `255.255.255.255` (broadcast).
 - Sent to **UDP port 67** (DHCP server side).
- Broadcast ensures **all DHCP servers** in the subnet can hear it.
- Contains a **transaction ID** to match responses.

2. DHCP Offer (Server → Broadcast)

- One or more DHCP servers respond with a DHCP Offer:
 - Contains:
 - **yiaddr (your IP address)** → proposed IP (e.g., `223.1.2.4`).
 - Subnet mask.
 - Lease time (e.g., `3600 sec = 1 hr`).
 - DHCP server ID (IP of server).
 - Same transaction ID as Discover.
- Sent as **broadcast** (`255.255.255.255`) so client receives it even without an assigned IP.
- Multiple servers may reply → client chooses one.

3. DHCP Request (Client → Broadcast)

- Client selects **one server's offer** and broadcasts a DHCP Request:
 - Confirms chosen IP address (echoes yiaddr).
 - Includes the selected server ID.
- Broadcast ensures **all servers know** which one was chosen.
- Prevents IP duplication (other servers withdraw their offers).

4. DHCP ACK (Server → Broadcast)

- Chosen server confirms with a DHCP Acknowledgment:
 - Reconfirms IP address, lease time, and configuration.
 - Client officially **uses the assigned IP address** for the lease duration.
-



Lease & Renewal

- IP address is given for a **lease time** (e.g., 1 hour, 1 day).
 - Before lease expires, client must **renew** (send a DHCP Request again).
 - If lease expires and not renewed → IP goes back into pool for reuse.
-



Limitation of DHCP

- Each time a host moves to a **new subnet**, it gets a **new IP address**.
- This breaks ongoing **TCP connections** (since TCP sessions depend on stable IPs).
- Solution in mobile networks: use **techniques beyond DHCP** (covered in Chapter 7).

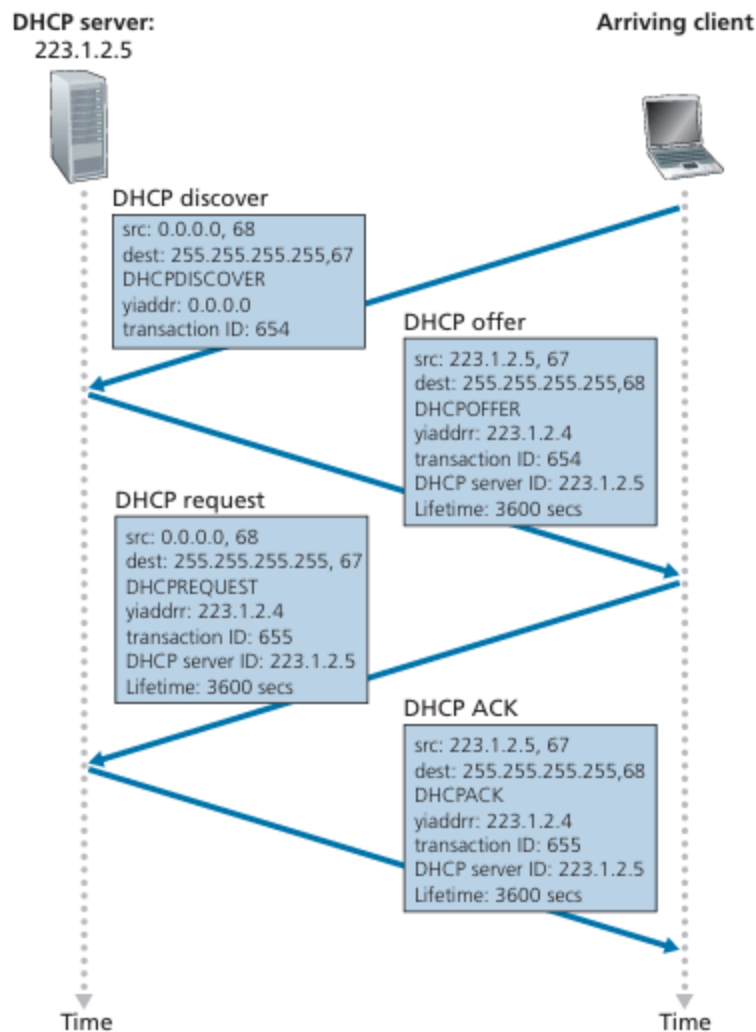


Figure 4.24 ♦ DHCP client-server interaction

Example Walkthrough (Figures 4.23 & 4.24)

1. A laptop joins subnet **223.1.2/24**.
2. Sends **DHCP Discover** (0.0.0.0 → 255.255.255.255).
3. DHCP server at **223.1.2.5** replies with **Offer**: IP = **223.1.2.4**, Lease = 3600 sec.
4. Client sends **Request** for **223.1.2.4**.
5. Server confirms with **ACK**.

6. Client now has:

- IP = 223.1.2.4
- Subnet mask = /24
- Gateway = 223.1.2.9
- DNS = server-configured
- Lease = 3600 sec

Summary

- DHCP = **automated, plug-and-play IP assignment**.
- Process = **Discover** → **Offer** → **Request** → **ACK (DORA)**.
- Provides IP, subnet mask, gateway, DNS, lease time.
- Works via **broadcasts**, often with **relay agents** if no local server.
- Limitation: causes IP changes when moving subnets → not suitable for seamless mobility.

👉 Would you like me to also create a **step-by-step timeline diagram** (with arrows & labels like in Fig. 4.24 but simplified) so you can memorize the **DORA exchange** quickly for exams?

Network Address Translation (NAT) – Detailed Explanation

Why NAT Exists

- Every **IP-capable device** needs an IP address.
 - With the explosion of **home/office LANs** (PCs, phones, smart TVs, IoT devices), allocating **globally unique IPs** to each device would quickly exhaust IPv4's 4.3 billion addresses.
 - **Solution:** NAT allows many private devices to share **one public IP address**.
-



Private vs Public Addresses

- **Private address ranges (RFC 1918):**
 - 10.0.0.0/8
 - 172.16.0.0/12
 - 192.168.0.0/16
 - These addresses **only work within local networks** (home, office).
 - Cannot be used on the public Internet (since millions of networks reuse them).
 - The NAT-enabled router translates between **private internal addresses** and **a single public IP**.
-



How NAT Works (Figure 4.25 Example)

Scenario

- Home network = 10.0.0.0/24.
- NAT router's WAN (public) IP = 138.76.29.7.
- Host (10.0.0.1) requests a webpage from 128.119.40.186 (port 80).

Step 1: Host Sends Request

- Host **10.0.0.1** chooses random source port **3345**.
- Sends datagram:
 - Src = **10.0.0.1:3345**
 - Dst = **128.119.40.186:80**

Step 2: NAT Translation at Router

- NAT router intercepts packet.
- Rewrites:
 - Src IP = **138.76.29.7** (router's public address).
 - Src Port = **5001** (new, unique port chosen by router).
- Updates **NAT translation table**:

WAN Side (Public) LAN Side (Private)

138.76.29.7:5001 10.0.0.1:3345

Step 3: Web Server Responds

- Web server replies:
 - Src = **128.119.40.186:80**
 - Dst = **138.76.29.7:5001**

Step 4: NAT Router Maps Back

- NAT looks up entry `138.76.29.7:5001` → `10.0.0.1:3345`.
 - Rewrites destination back to private address.
 - Forwards to host `10.0.0.1`.
-

Key Features

- **Port numbers used in NAT table** → allow router to distinguish multiple connections from many devices.
 - With 16-bit ports (~65,000), one public IP can support **tens of thousands of simultaneous connections**.
 - To the outside world: entire LAN looks like **one device** (the NAT router).
-

Issues with NAT

Technical Problems

1. **Servers inside NAT:**
 - Servers usually listen on well-known ports (e.g., HTTP = 80).
 - NAT blocks unsolicited incoming connections, so running a server behind NAT is tricky.
 - Peer-to-Peer (P2P) apps (e.g., torrents, VoIP) struggle.
 - Solution: NAT traversal (STUN, TURN, ICE, UPnP).
2. **Port overloading:**
 - NAT reuses ports in creative ways → complicates protocols that embed IP/port info in payloads.

Philosophical Problems

- **End-to-End Principle:**
 - Internet's design assumed **hosts communicate directly** using unique IPs.
 - NAT breaks this: routers (Layer 3 devices) now **modify IP addresses and ports** → mixing network and transport layer logic.
 - Critics argue NAT **violates clean layering** and complicates applications.
-

Summary

- NAT enables private home/office networks to connect many devices using a **single public IP**.
 - Works by mapping **private IP:port** ↔ **public IP:port** in a translation table.
 - Efficient but creates **challenges for servers, P2P apps, and end-to-end transparency**.
 - Despite drawbacks, NAT is extremely common and essential in IPv4 due to **address scarcity**.
-

👉 Do you want me to also make a **step-by-step NAT translation table example** (multiple hosts → one public IP) so you can clearly see how simultaneous connections are tracked?